Normalização de Bancos de Dados

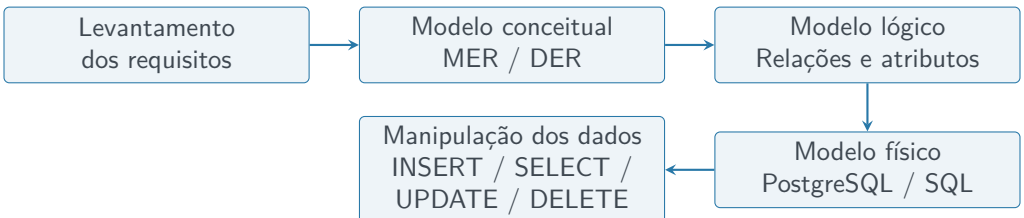
Da Modelagem Relacional à Implementação SQL



Prof. Dr. João Choma
Universidade Estadual de Maringá - DIN/CTC
github.com/JoaoChoma

Banco de Dados I ▪ Formas normais e Supabase

O que já aprendemos?



Conhecimentos prévios e nova pergunta



Aprendemos a:

- Identificar entidades e atributos.
- Estabelecer relacionamentos e cardinalidades.
- Transformar relacionamentos em tabelas.
- Definir chaves primárias e estrangeiras.
- Criar e alterar tabelas utilizando SQL.
- Inserir, consultar, atualizar e excluir registros.
- Definir restrições para preservar a integridade dos dados.

Nova pergunta: *Uma estrutura pode funcionar corretamente em SQL e, mesmo assim, apresentar problemas de modelagem?*

Sim.

Uma tabela pode funcionar e estar mal projetada?



Uma empresa armazena informações sobre alunos e cursos:

id_aluno	nome_aluno	id_curso	nome_curso	coordenador
1	Ana	10	Sistemas de Informação	Carlos
2	Bruno	10	Sistemas de Informação	Carlos
3	Maria	20	Ciência da Computação	Fernanda
4	Pedro	10	Sistemas de Informação	Carlos
5	Julia	20	Ciência da Computação	Fernanda

O banco funciona. Mas a estrutura é adequada?

Implementando a tabela no Supabase



```sql
CREATE TABLE aluno_curso (
    id_aluno BIGINT PRIMARY KEY,
    nome_aluno VARCHAR(100) NOT NULL,
    id_curso BIGINT NOT NULL,
    nome_curso VARCHAR(100) NOT NULL,
    coordenador VARCHAR(100) NOT NULL
);
```

```sql
INSERT INTO aluno_curso
(id_aluno, nome_aluno, id_curso, nome_curso, coordenador)
VALUES
(1, 'Ana', 10, 'Sistemas de Informação', 'Carlos'),
(2, 'Bruno', 10, 'Sistemas de Informação', 'Carlos'),
(3, 'Maria', 20, 'Ciência da Computação', 'Fernanda'),
(4, 'Pedro', 10, 'Sistemas de Informação', 'Carlos'),
(5, 'Julia', 20, 'Ciência da Computação', 'Fernanda');
```

```sql
SELECT * FROM aluno_curso;
```

Identificando problemas



id_aluno	nome_aluno	id_curso	nome_curso	coordenador
1	Ana	10	Sistemas de Informação	Carlos
2	Bruno	10	Sistemas de Informação	Carlos
3	Maria	20	Ciência da Computação	Fernanda
4	Pedro	10	Sistemas de Informação	Carlos

- **Redundância:** informações do curso se repetem para cada aluno.
- **Anomalia de atualização:** mudar coordenador exige alterar várias linhas.
- **Anomalia de inserção:** não é possível cadastrar curso sem aluno nessa estrutura.
- **Anomalia de exclusão:** excluir o último aluno pode apagar os dados do curso.

Demonstrando uma anomalia com SQL



Carlos deixou de coordenar Sistemas de Informação.

```sql
UPDATE aluno_curso
SET coordenador = 'Roberto'
WHERE id_aluno = 1;
```

```sql
SELECT *
FROM aluno_curso
WHERE id_curso = 10;
```

id_aluno	nome_aluno	nome_curso	coordenador
1	Ana	Sistemas de Informação	Roberto
2	Bruno	Sistemas de Informação	Carlos
4	Pedro	Sistemas de Informação	Carlos

Pergunta: Quem é o coordenador do curso?

Corrigir o UPDATE resolve a modelagem?



Podemos atualizar todas as linhas:

```sql
UPDATE aluno_curso
SET coordenador = 'Roberto'
WHERE id_curso = 10;
```

Isso corrige os registros **naquele momento**, mas **não elimina a redundância estrutural**.

A informação continua armazenada repetidamente.

O que é normalização?



A **normalização** organiza as relações de um banco de dados para reduzir redundâncias problemáticas e evitar anomalias.

O processo utiliza regras chamadas **Formas Normais**.

Busca:

1. Organizar adequadamente os atributos.
2. Identificar dependências entre os dados.
3. Reduzir redundâncias que provocam inconsistências.
4. Preservar a integridade das informações.
5. Facilitar a manutenção dos registros.

*Normalizar **não** é simplesmente criar mais tabelas. A decomposição deve respeitar as dependências entre os atributos.*

Em qual etapa aplicamos a normalização?



A normalização é aplicada principalmente na elaboração/revisão do **modelo lógico**. Também é possível revisar um banco já implementado, modificando estruturas e migrando dados.

O que é dependência funcional?



Considere:

```
ALUNO(id_aluno, nome_aluno, id_curso, nome_curso, coordenador)
```

```
id_aluno → nome_aluno
id_aluno → id_curso
id_curso → nome_curso
id_curso → coordenador
```

Leitura: `id_aluno` $\rightarrow$ `nome_aluno` significa que o identificador do aluno determina seu nome.

Definição: X $\rightarrow$ Y significa que, se dois registros têm o mesmo valor de X, devem ter o mesmo valor de Y.

Premissa de negócio: cada aluno está matriculado em um único curso; cada curso possui um nome e um coordenador no exemplo.

Dependência transitiva



```
id_aluno → id_curso
id_curso → nome_curso
```

Consequentemente:

```
id_aluno → nome_curso
```

```
id_aluno
    |
    ▼
id_curso
    |
    ▼
nome_curso
```

O nome do curso está associado ao aluno **por meio** do identificador do curso.

Isso exemplifica uma **dependência transitiva**, relevante para a 3FN.

Primeira Forma Normal (1FN)



Uma relação está na **1FN** quando seus atributos possuem valores atômicos no domínio considerado e não existem grupos repetitivos de atributos.

id_cliente	nome	telefones
1	Ana	9999-1111, 9888-2222
2	Bruno	9777-3333

A coluna `telefones` guarda vários números em um único campo textual.

Problema: para o domínio adotado, cada telefone deve ser representado individualmente.

Estrutura problemática em SQL



```sql
CREATE TABLE cliente_nao_normalizado (
    id_cliente BIGINT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    telefones VARCHAR(200)
);
```

```sql
INSERT INTO cliente_nao_normalizado
(id_cliente, nome, telefones)
VALUES
(1, 'Ana', '9999-1111, 9888-2222'),
(2, 'Bruno', '9777-3333');
```

```sql
SELECT * FROM cliente_nao_normalizado;
```

Por que essa estrutura é problemática?



Guardar telefones como lista textual dificulta:

- Adicionar um telefone individual.
- Excluir um telefone específico.
- Validar individualmente os números.
- Consultar clientes por um número determinado.

Discussão: como representar corretamente uma relação de um cliente com vários telefones?

Aplicando a 1FN



```
CLIENTE(id_cliente PK, nome)

TELEFONE(id_telefone PK, numero, id_cliente FK)

CLIENTE 1 ———————— N TELEFONE
```

```sql
CREATE TABLE cliente (
    id_cliente BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome VARCHAR(100) NOT NULL
);

CREATE TABLE telefone (
    id_telefone BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    numero VARCHAR(20) NOT NULL,
    id_cliente BIGINT NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
);
```

Inserindo dados após a 1FN



```sql
INSERT INTO cliente (nome)
VALUES ('Ana'), ('Bruno');
```

```sql
SELECT * FROM cliente;
```

Considerando Ana com ID 1 e Bruno com ID 2:

```sql
INSERT INTO telefone (numero, id_cliente)
VALUES
('9999-1111', 1),
('9888-2222', 1),
('9777-3333', 2);
```

Cada telefone está armazenado individualmente.

Nota: atomicidade depende do domínio e das operações necessárias; não significa dividir todo texto em partes menores.

Segunda Forma Normal (2FN)



Uma relação está na **2FN** quando:

1. Está na 1FN.
2. Nenhum atributo **não-primo** depende funcionalmente de apenas parte de uma chave candidata.

Atributo primo: atributo que pertence a alguma chave candidata.

Uma **chave primária composta** utiliza dois ou mais atributos para identificar um registro.

A dependência parcial fica evidente quando há chave composta.

Exemplo: itens de pedidos



```
ITEM_PEDIDO(id_pedido, id_produto, nome_produto, preco_produto, quantidade)

PK: (id_pedido, id_produto)
```

id_pedido	id_produto	nome_produto	preco_produto	quantidade
1	10	Mouse	80,00	2
1	20	Teclado	150,00	1
2	10	Mouse	80,00	3
3	10	Mouse	80,00	1

O nome do produto depende da combinação pedido + produto ou apenas de `id_produto`?

Dependências parciais

```
(id_pedido, id_produto) → quantidade

id_produto → nome_produto
id_produto → preco_produto
```

```
CHAVE PRIMÁRIA COMPOSTA
 (id_pedido, id_produto)
              │
              ▼
         quantidade

        id_produto
             ├──► nome_produto
             └──► preco_produto
```

`nome_produto` e `preco_produto` dependem de **parte** da chave composta: violação da **2FN**.

Uma distinção sobre preços



No exemplo, `preco_produto` representa o **preço cadastral atual** e depende de `id_produto`.

Mas o preço efetivamente cobrado em uma venda pode variar entre pedidos.

Nesse caso:

```
(id_pedido, id_produto) → preco_unitario_venda
```

Logo, `preco_unitario_venda` deve permanecer em `ITEM_PEDIDO`.

A normalização depende do significado dos atributos e das regras de negócio.

Corrigindo a 2FN



```
PRODUTO(id_produto PK, nome_produto, preco_atual)

PEDIDO(id_pedido PK)

ITEM_PEDIDO(id_pedido PK, FK, id_produto PK, FK, quantidade, preco_unitario_venda)
```

```
PEDIDO 1 ———————— N ITEM_PEDIDO
PRODUTO 1 ———————— N ITEM_PEDIDO
```

A chave de `ITEM_PEDIDO` permanece composta, mas os atributos ficam nas relações correspondentes às suas dependências.

Implementando a 2FN no Supabase



```sql
CREATE TABLE produto (
    id_produto BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome_produto VARCHAR(100) NOT NULL,
    preco_atual NUMERIC(10,2) NOT NULL
        CHECK (preco_atual >= 0)
);

CREATE TABLE pedido (
    id_pedido BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY
);
```

Chave composta e referências



```sql
CREATE TABLE item_pedido (
    id_pedido BIGINT NOT NULL,
    id_produto BIGINT NOT NULL,
    quantidade INTEGER NOT NULL CHECK (quantidade > 0),
    preco_unitario_venda NUMERIC(10,2) NOT NULL
        CHECK (preco_unitario_venda >= 0),

    PRIMARY KEY (id_pedido, id_produto),

    FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido),
    FOREIGN KEY (id_produto) REFERENCES produto(id_produto)
);
```

`PRIMARY KEY (id_pedido, id_produto)` define a **chave composta**.

Com esse modelo, o mesmo produto pode estar em pedidos diferentes, mas aparece uma única vez por pedido.

Terceira Forma Normal (3FN)



Uma relação está na **3FN** quando:

1. Está na 2FN.
2. Para toda dependência funcional não trivial $X \rightarrow A$, **X é uma superchave** ou **A é um atributo primo**.

Simplificação didática: frequentemente a 3FN elimina dependências transitivas inadequadas entre atributos não-chave.

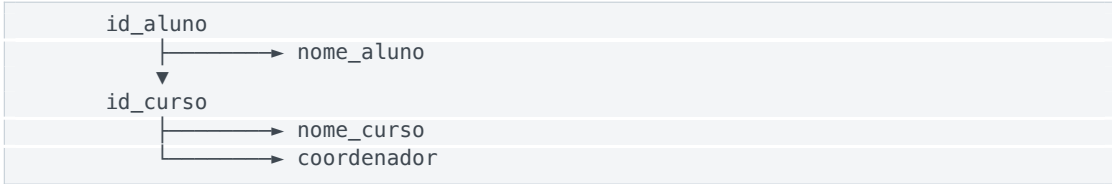
Retomando:

```
ALUNO_CURSO(id_aluno PK, nome_aluno, id_curso, nome_curso, coordenador)
```

Identificando o problema da 3FN



```
id_aluno → nome_aluno
id_aluno → id_curso
id_curso → nome_curso
id_curso → coordenador
```



`id_curso` **não é superchave** de `ALUNO_CURSO`.

`nome_curso` e `coordenador` **não são atributos primos**.

Portanto, essas dependências violam a 3FN.

Modelo lógico na 3FN



```
CURSO(id_curso PK, nome_curso, coordenador)

ALUNO(id_aluno PK, nome_aluno, id_curso FK)

CURSO 1 ———————— N ALUNO
```

Agora:

```
id_curso → nome_curso, coordenador
id_aluno → nome_aluno, id_curso
```

Cada informação está armazenada na relação correspondente à sua dependência.

Implementando a 3FN em SQL



```sql
CREATE TABLE curso (
    id_curso BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome_curso VARCHAR(100) NOT NULL,
    coordenador VARCHAR(100) NOT NULL
);

CREATE TABLE aluno (
    id_aluno BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome_aluno VARCHAR(100) NOT NULL,
    id_curso BIGINT NOT NULL,
    FOREIGN KEY (id_curso) REFERENCES curso(id_curso)
);
```

Inserindo cursos e alunos



```sql
INSERT INTO curso (nome_curso, coordenador)
VALUES
('Sistemas de Informação', 'Carlos'),
('Ciência da Computação', 'Fernanda');
```

```sql
SELECT * FROM curso;
```

Se os IDs gerados forem 1 e 2, respectivamente:

```sql
INSERT INTO aluno (nome_aluno, id_curso)
VALUES
('Ana', 1),
('Bruno', 1),
('Maria', 2),
('Pedro', 1),
('Julia', 2);
```

As informações de cada curso ficam armazenadas uma única vez.

UPDATE antes e depois da normalização



Antes: modificar os registros que repetem o coordenador.

```
UPDATE aluno_curso
SET coordenador = 'Roberto'
WHERE id_curso = 10;
```

Depois: modificar somente o registro do curso.

```
UPDATE curso
SET coordenador = 'Roberto'
WHERE nome_curso = 'Sistemas de Informação';
```

Resultado: os dados dos alunos não precisam ser atualizados individualmente.

Comparação das formas normais



Forma	Problema	Estratégia
1FN	Valores multivalorados e grupos repetitivos	Valores atômicos e ausência de grupos repetitivos
2FN	Dependência parcial de chave candidata	Separar atributos dependentes de parte da chave
3FN	Dependências funcionais que violam a 3FN	Separar atributos conforme suas dependências

Progressão cumulativa

Relação não normalizada $\rightarrow$ 1FN: valores atômicos $\rightarrow$ 2FN: sem dependências parciais $\rightarrow$ 3FN: condições formais da 3FN

As formas normais são cumulativas.



Os alunos já conhecem:

```
PRIMARY KEY
FOREIGN KEY
NOT NULL
UNIQUE
CHECK
DEFAULT
```

Essas restrições controlam a integridade dos registros.

Normalização e restrições de integridade não são a mesma coisa.

Restrições não garantem normalização



```sql
CREATE TABLE aluno_curso (
    id_aluno BIGINT PRIMARY KEY,
    nome_aluno VARCHAR(100) NOT NULL,
    id_curso BIGINT NOT NULL,
    nome_curso VARCHAR(100) NOT NULL
);
```

A tabela tem:

- Chave primária.
- Campos obrigatórios.
- Tipos definidos.

Mesmo assim, pode apresentar problemas de normalização.

*Restrições SQL controlam **quais dados são aceitos**; a normalização orienta **onde os dados devem ser armazenados**.*

Atividade: funcionários e departamentos



```sql
CREATE TABLE funcionario_departamento (
    id_funcionario BIGINT PRIMARY KEY,
    nome_funcionario VARCHAR(100) NOT NULL,
    salario NUMERIC(10,2) NOT NULL,
    id_departamento BIGINT NOT NULL,
    nome_departamento VARCHAR(100) NOT NULL,
    gerente_departamento VARCHAR(100) NOT NULL
);
```

```sql
INSERT INTO funcionario_departamento
VALUES
(1, 'Ana', 3500.00, 10, 'TI', 'Carlos'),
(2, 'Bruno', 4200.00, 10, 'TI', 'Carlos'),
(3, 'Maria', 3100.00, 20, 'Financeiro', 'Fernanda'),
(4, 'Pedro', 3900.00, 10, 'TI', 'Carlos'),
(5, 'Julia', 3700.00, 20, 'Financeiro', 'Fernanda');
```

Exercício 1: analisando a estrutura



1. Quais informações são repetidas?
2. Quais são as dependências funcionais?
3. A tabela está na 1FN? Justifique.
4. A tabela está na 2FN? Justifique.
5. A tabela está na 3FN? Justifique.
6. Qual anomalia poderia ocorrer se o gerente de TI fosse alterado?
7. Qual anomalia poderia ocorrer ao excluir o único funcionário de um departamento?

Atenção: a tabela possui uma **chave primária simples**; considere isso na análise da 2FN.

Exercício 2: normalizando



A partir da tabela anterior:

1. Identifique as entidades.
2. Defina seus atributos.
3. Estabeleça as chaves primárias.
4. Identifique o relacionamento e sua cardinalidade.
5. Construa o modelo lógico relacional normalizado até a 3FN.

Resultado esperado:

```
DEPARTAMENTO(id_departamento PK, nome_departamento, gerente_departamento)

FUNCIONARIO(id_funcionario PK, nome_funcionario, salario, id_departamento FK)
```

Exercício 3: implementando no Supabase



Implemente o modelo normalizado utilizando PostgreSQL.

1. Criar as tabelas com `CREATE TABLE`.
2. Definir as chaves primárias.
3. Definir a chave estrangeira.
4. Utilizar `NOT NULL` nos atributos obrigatórios.
5. Inserir os departamentos.
6. Inserir os funcionários.
7. Consultar os registros com `SELECT`.
8. Alterar o gerente do departamento de TI.
9. Verificar se os dados dos funcionários permaneceram consistentes.

Não é necessário utilizar `JOIN` nesta atividade.

O que aprendemos?



- Uma tabela pode funcionar e apresentar problemas estruturais.
- A redundância pode provocar inconsistências.
- Dependências funcionais ajudam a identificar problemas de modelagem.
- A **1FN** trata de valores atômicos e grupos repetitivos.
- A **2FN** elimina dependências parciais inadequadas.
- A **3FN** impõe condições formais às dependências funcionais.
- A normalização pode exigir decompor uma relação em várias.
- Chaves primárias e estrangeiras implementam os relacionamentos resultantes.